INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	8 de noviembre de 2026
Enlace de Git	https://github.com/Danilrox/cafeteria-bd
Nombres y apellidos del estudiantes	Jose Daniel Maldonado Rodriguez

1. TÍTULO

“Diseño e implementación de una base de datos relacional para la gestión de ventas, pedidos e inventario de una cafetería”

2. CONTEXTO DEL PROBLEMA

Una cafetería ofrece bebidas preparadas al momento (café espresso, capuchino, latte y chocolate caliente) y productos de repostería (porciones de pastel, brownies, rollos de canela, donas, alfajores, cupcakes y galletas). Además de las ventas rápidas en el mostrador, muchas de ellas anónimas, el local recibe pedidos por encargo de mayor volumen, que requieren registrar un anticipo, una fecha de entrega y el estado de elaboración. Para producir estos alimentos, el negocio compra insumos como leche, café, harina, huevos, mantequilla y chocolate a distintos proveedores, y emite facturas cuando el cliente las solicita.

Sin una base de datos, esta información suele llevarse en cuadernos, hojas de cálculo sueltas o de memoria. Eso genera datos dispersos y duplicados, no permite saber con certeza cuántos insumos quedan ni cuándo reabastecerlos, y dificulta conocer qué cantidad de cada insumo consume un producto (su receta), lo que impide planificar la producción y calcular costos. Los pedidos con anticipo pueden perderse o quedar sin seguimiento, los totales de ventas, compras y facturas pueden no coincidir, y no existe un historial confiable de las compras realizadas a cada proveedor.

El problema afecta al propietario o administrador, que necesita información correcta para decidir qué comprar y cuánto producir; al personal de caja y cocina, que debe consultar la disponibilidad de productos y el estado de los pedidos; a los clientes, que esperan entregas puntuales y comprobantes correctos; y a los proveedores, con quienes se mantiene una relación comercial que debe quedar registrada.

Por ello se deben gestionar de forma ordenada los datos de clientes, proveedores, insumos y sus existencias, productos con su precio y stock, recetas, compras con su detalle, ventas con su detalle y

método de pago, pedidos (anticipo, fecha de entrega y estado) y facturas (subtotal, IVA y total). Hacerlo correctamente es importante porque garantiza la integridad de los montos, permite rastrear cada transacción y ofrece una base confiable para la toma de decisiones. En la facturación se consideró la tarifa general del IVA del 15 % que el SRI mantiene vigente en Ecuador [6].

3. OBJETIVOS

3.1. Objetivo General

Diseñar e implementar una base de datos relacional en MySQL, mediante el modelado entidad-relación, el modelo relacional y el lenguaje SQL, para gestionar de forma integral las ventas, los pedidos, la facturación, las compras y el inventario de insumos de una cafetería, mejorando el control y la consistencia de su información.

3.2. Objetivos Específicos

- Elaborar el diagrama Entidad-Relación de la cafetería y derivar de él el modelo relacional, identificando entidades, atributos, relaciones, cardinalidades y claves primarias y foráneas.
- Construir el esquema físico en MySQL (sentencias DDL), poblarlo con datos de ejemplo coherentes (DML) e implementar cinco consultas formuladas en álgebra relacional y traducidas a SQL, que demuestren el funcionamiento de la base de datos.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

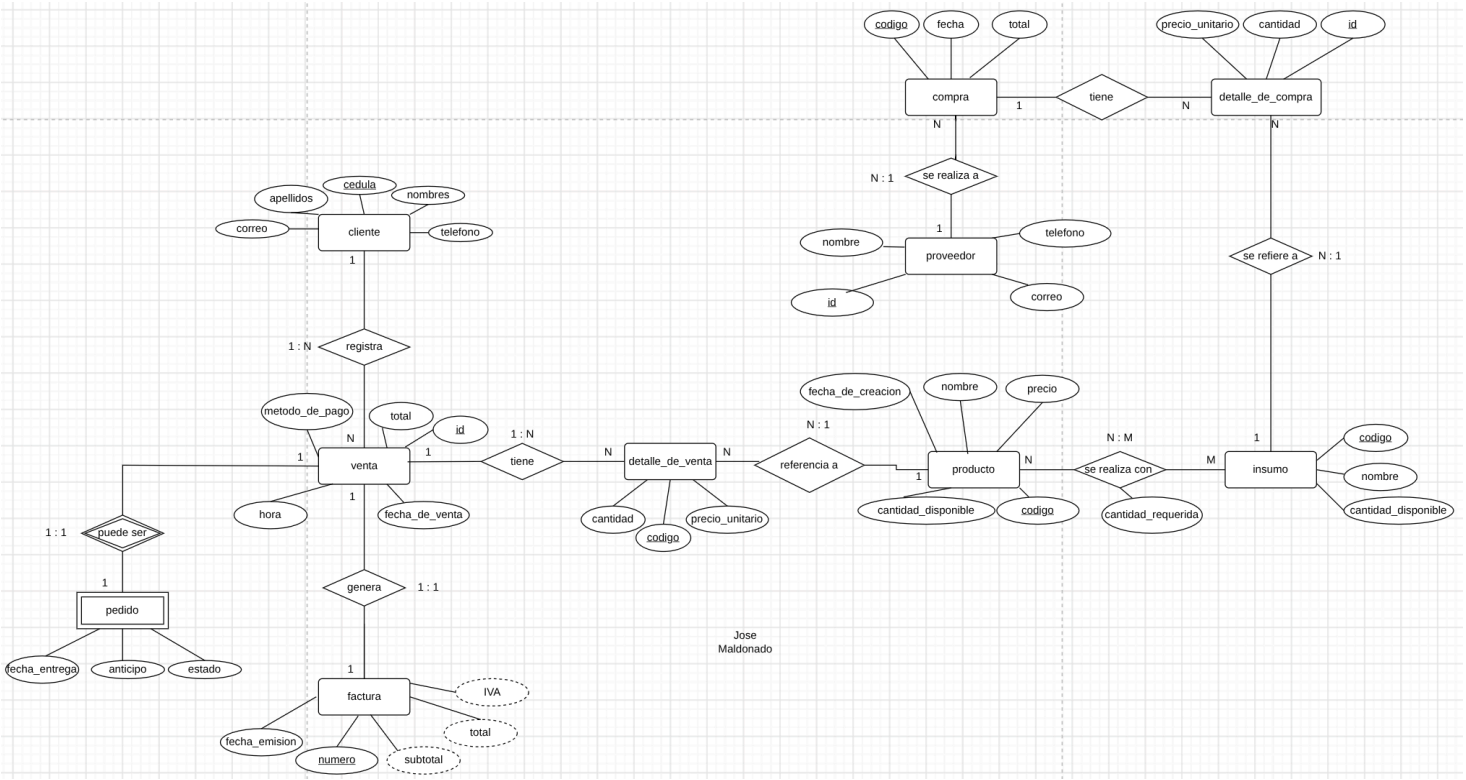


Figura 1. Diagrama Entidad-Relacion del sistema de gestion de cafeteria

Link drawio:
https://drive.google.com/file/d/1x5NUexeyrqrBJFi4KJPAE6EzKcKzOXYW/view?usp=sharing

4.2. Modelo Relacional

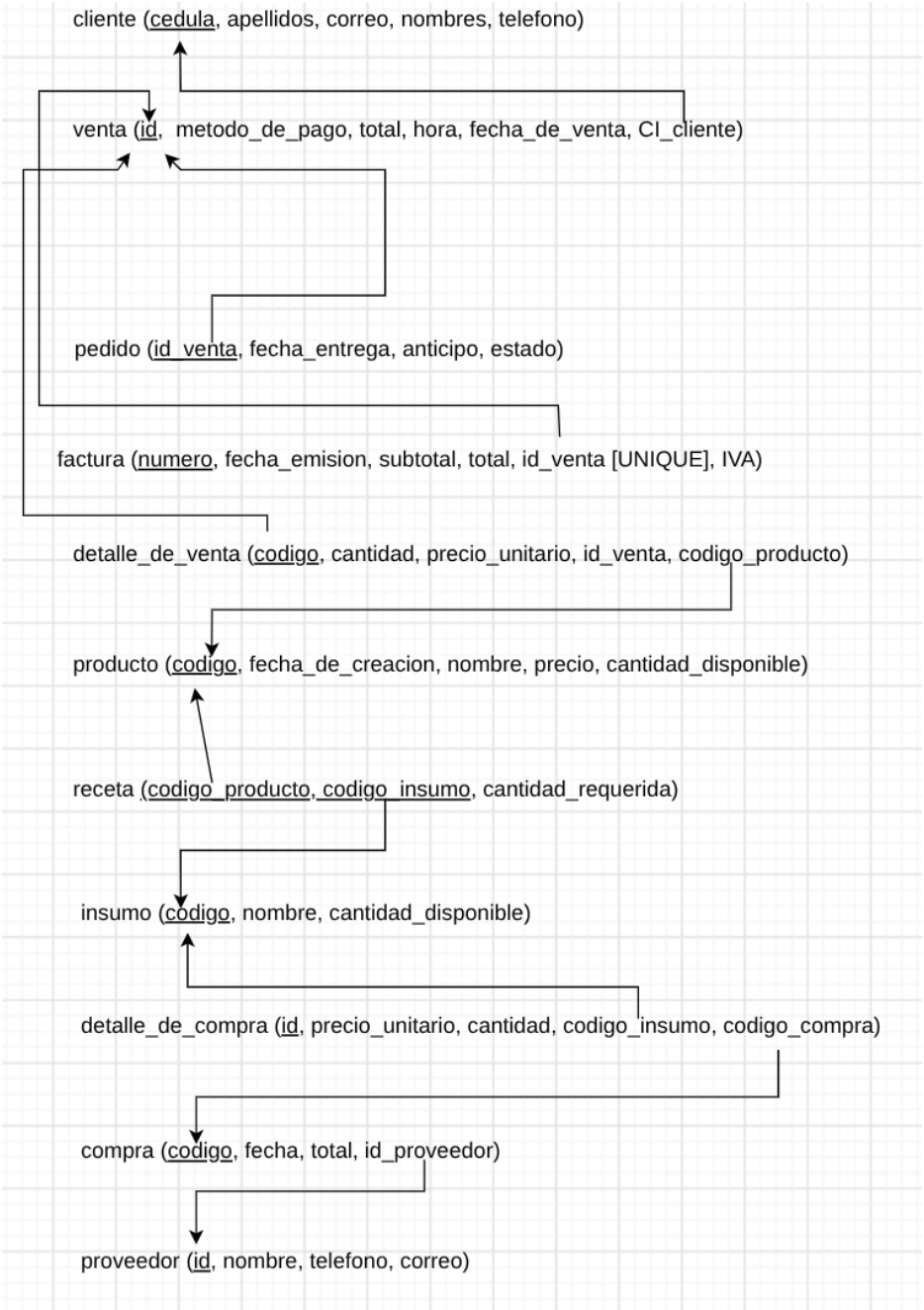


Figura 2. Modelo relacional del sistema de gestión de la cafetería (las flechas van de la llave foránea a la tabla referenciada).

Link de drawio:
https://drive.google.com/file/d/1v8SRWoD0MfejYMtDeaVqIpwTF59f4Def/view?usp=sharing

5. CÓDIGO SQL

5.1. Creación de Tablas (DDL)

Se presentan siete de las once tablas, representativas de las distintas situaciones del modelo: una entidad simple (cliente), una con restricciones CHECK y valor por defecto (producto), una tabla de llave compuesta que resuelve la relación N:M (receta), una con llave foránea que admite NULL (venta), una entidad débil (pedido), una relación 1:1 con UNIQUE (factura) y una tabla de detalle con dos llaves foráneas (detalle_de_venta).

```
CREATE DATABASE IF NOT EXISTS cafeteria
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE cafeteria;
```

```
CREATE TABLE cliente (
    cedula     VARCHAR(10)  NOT NULL,
    apellidos  VARCHAR(80)  NOT NULL,
    nombres    VARCHAR(80)  NOT NULL,
    correo     VARCHAR(100),
    telefono   VARCHAR(15),
    PRIMARY KEY (cedula)
) ENGINE=InnoDB;
```

```
CREATE TABLE producto (
    codigo               INT           NOT NULL AUTO_INCREMENT,
    fecha_de_creacion    DATE          NOT NULL,
    nombre               VARCHAR(100)  NOT NULL,
    precio               DECIMAL(10,2) NOT NULL,
    cantidad_disponible  INT           NOT NULL DEFAULT 0,
    PRIMARY KEY (codigo),
    CONSTRAINT chk_producto_precio CHECK (precio >= 0),
    CONSTRAINT chk_producto_cantidad CHECK (cantidad_disponible >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE receta (
    codigo_producto     INT           NOT NULL,
    codigo_insumo       INT           NOT NULL,
    cantidad_requerida  DECIMAL(10,3) NOT NULL,
    PRIMARY KEY (codigo_producto, codigo_insumo),
    CONSTRAINT fk_receta_producto
        FOREIGN KEY (codigo_producto) REFERENCES producto (codigo)
        ON DELETE CASCADE,
    CONSTRAINT fk_receta_insumo
        FOREIGN KEY (codigo_insumo) REFERENCES insumo (codigo)
        ON DELETE RESTRICT,
    CONSTRAINT chk_receta_cantidad CHECK (cantidad_requerida > 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE venta (
    id              INT           NOT NULL AUTO_INCREMENT,
    metodo_de_pago  VARCHAR(20)   NOT NULL,
    total           DECIMAL(10,2) NOT NULL DEFAULT 0,
    hora            TIME          NOT NULL,
    fecha_de_venta  DATE          NOT NULL,
    CI_cliente      VARCHAR(10)   NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_venta_cliente
        FOREIGN KEY (CI_cliente) REFERENCES cliente (cedula)
        ON DELETE RESTRICT,
    CONSTRAINT chk_venta_total CHECK (total >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE pedido (
    id_venta       INT           NOT NULL,
    fecha_entrega  DATE          NOT NULL,
    anticipo       DECIMAL(10,2) NOT NULL DEFAULT 0,
    estado         VARCHAR(20)   NOT NULL DEFAULT 'recibido',
    PRIMARY KEY (id_venta),
    CONSTRAINT fk_pedido_venta
        FOREIGN KEY (id_venta) REFERENCES venta (id)
        ON DELETE CASCADE,
    CONSTRAINT chk_pedido_anticipo CHECK (anticipo >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE factura (
    numero         VARCHAR(17)   NOT NULL,
    fecha_emision  DATE          NOT NULL,
    subtotal       DECIMAL(10,2) NOT NULL,
    iva            DECIMAL(10,2) NOT NULL,
    total          DECIMAL(10,2) NOT NULL,
    id_venta       INT           NOT NULL,
    PRIMARY KEY (numero),
    UNIQUE KEY uq_factura_venta (id_venta),
    CONSTRAINT fk_factura_venta
        FOREIGN KEY (id_venta) REFERENCES venta (id)
        ON DELETE RESTRICT,
    CONSTRAINT chk_factura_valores CHECK (subtotal >= 0 AND iva >= 0 AND total >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE detalle_de_venta (
    codigo           INT           NOT NULL AUTO_INCREMENT,
    cantidad         INT           NOT NULL,
    precio_unitario  DECIMAL(10,2) NOT NULL,
    id_venta         INT           NOT NULL,
    codigo_producto  INT           NOT NULL,
    PRIMARY KEY (codigo),
    CONSTRAINT fk_detventa_venta
        FOREIGN KEY (id_venta) REFERENCES venta (id)
        ON DELETE CASCADE,
    CONSTRAINT fk_detventa_producto
        FOREIGN KEY (codigo_producto) REFERENCES producto (codigo)
        ON DELETE RESTRICT,
    CONSTRAINT chk_detventa_cantidad CHECK (cantidad > 0),
    CONSTRAINT chk_detventa_precio CHECK (precio_unitario >= 0)
) ENGINE=InnoDB;
```

5.2. Inserción de Datos (DML)

```
USE cafeteria;

INSERT INTO cliente (cedula, apellidos, nombres, correo, telefono) VALUES
('1712345678', 'Muñoz Paredes',  'Ana Lucía',   'ana.munoz@correo.com',
'0991234567'),
('1723456789', 'Torres Vega',    'Carlos Iván', 'carlos.torres@correo.com',
'0982345678'),
('0912345678', 'Salazar Rivera', 'María José',  'mj.salazar@correo.com',
'0973456789'),
('1104567890', 'Ortega León',    'Diego',       NULL,
'0964567890'),
('1801234567', 'Cevallos Mena',  'Paola',       'paola.cevallos@correo.com', NULL);
```

```
INSERT INTO producto (codigo, fecha_de_creacion, nombre, precio, cantidad_disponible)
VALUES
(1,  '2026-01-15', 'Café espresso',                          1.50,  0),
(2,  '2026-01-15', 'Capuchino',                              2.25,  0),
(3,  '2026-01-15', 'Latte',                                  2.50,  0),
(4,  '2026-01-15', 'Chocolate caliente',                     2.00,  0),
(5,  '2026-02-10', 'Pastel de chocolate con manjar (porción)', 3.50, 12),
(6,  '2026-02-10', 'Pastel de vainilla con fresa (porción)',   3.50, 10),
(7,  '2026-03-05', 'Brownie',                                2.00, 15),
(8,  '2026-11-08', 'Rollo de canela',                        2.50, 12),
(9,  '2026-11-08', 'Pastel de zanahoria (porción)',          3.50,  8),
(10, '2026-11-08', 'Dona',                                   1.25, 20),
(11, '2026-11-08', 'Alfajor de maicena',                     1.00, 25),
(12, '2026-11-08', 'Alfajor de chocolate',                   1.25, 25),
(13, '2026-11-08', 'Cupcake de chocolate',                   2.00, 15),
(14, '2026-11-08', 'Pastel de banano (porción)',             3.25,  8),
(15, '2026-11-08', 'Galleta de avena',                       1.00, 30);
```

```
INSERT INTO venta (id, metodo_de_pago, total, hora, fecha_de_venta, CI_cliente)
VALUES
(1, 'efectivo',       6.50, '09:15:00', '2026-11-06', NULL),
(2, 'transferencia',  6.00, '16:40:00', '2026-11-06', '1712345678'),
(3, 'transferencia', 54.00, '11:00:00', '2026-11-07', '1723456789'),
(4, 'efectivo',       3.50, '08:30:00', '2026-11-07', NULL),
(5, 'efectivo',       5.25, '09:05:00', '2026-11-08', NULL),
(6, 'tarjeta',        9.75, '10:20:00', '2026-11-08', '0912345678'),
(7, 'transferencia', 66.00, '12:00:00', '2026-11-08', '1104567890'),
(8, 'efectivo',       5.75, '15:30:00', '2026-11-08', NULL);
```

Las ventas 3 y 7 son pedidos (con anticipo y fecha de entrega) y la venta 2 tiene factura; las ventas 1, 4, 5 y 8 son ventas anónimas del mostrador, con CI_cliente nulo. Los montos de factura incluyen el IVA del 15 % [6]: por ejemplo, el total de 6,00 se descompone en un subtotal de 5,22 y un IVA de 0,78.

6. CONSULTAS DE ÁLGEBRA RELACIONAL Y SQL

Consulta 1 — Selección y proyección (operadores σ y π)

Descripción: obtener el nombre, el precio y el stock de los productos que tienen existencias bajas, es decir, más de 0 y menos de 10 unidades, para decidir cuáles conviene volver a preparar. Se excluyen las bebidas, que se preparan al momento y tienen stock 0.

Álgebra relacional:

```
π nombre, precio, cantidad_disponible ( σ cantidad_disponible > 0 and
cantidad_disponible < 10 ( PRODUCTO ) )
```

SQL:

```
SELECT nombre, precio, cantidad_disponible
FROM   producto
WHERE  cantidad_disponible > 0
  AND  cantidad_disponible < 10;
```

Resultado esperado: dos productos con 8 unidades en stock.

nombre	precio	cantidad_disponible
Pastel de zanahoria (porción)	3,50	8
Pastel de banano (porción)	3,25	8

Consulta 2 — Reunión de tres relaciones (operador ⋈)

Descripción: listar, para cada pedido, los datos del cliente que lo encargó, la fecha de entrega, el anticipo pagado, el saldo pendiente (total de la venta menos anticipo) y el estado del pedido. Combina las tablas cliente, venta y pedido mediante sus llaves foráneas y calcula el saldo con una proyección generalizada.

Álgebra relacional:

```
π cedula, apellidos, nombres, fecha_entrega, anticipo, saldo ← total − anticipo,
estado ( CLIENTE ⋈ cedula = CI_cliente VENTA ⋈ id = id_venta PEDIDO )
```

SQL:

```
SELECT c.cedula, c.apellidos, c.nombres,
       p.fecha_entrega, p.anticipo,
       v.total - p.anticipo AS saldo,
       p.estado
FROM   cliente c
JOIN   venta  v ON v.CI_cliente = c.cedula
JOIN   pedido p ON p.id_venta   = v.id
ORDER BY p.fecha_entrega;
```

Resultado esperado: los dos pedidos registrados, con el saldo que el cliente debe pagar al retirarlos.

cedula	apellidos	nombres	fecha_entrega	anticipo	saldo	estado
1723456789	Torres Vega	Carlos Iván	2026-11-09	20,00	34,00	en preparación
1104567890	Ortega León	Diego	2026-11-12	30,00	36,00	recibido

Consulta 3 — Diferencia (operador −)

Descripción: identificar los productos del catálogo que nunca se han vendido, restando del conjunto de todos los códigos de producto el conjunto de códigos que aparecen en algún detalle de venta. Sirve para detectar productos que no rotan.

Álgebra relacional:

```
π codigo, nombre ( PRODUCTO ⋈ ( π codigo ( PRODUCTO ) − ρ codigo ← codigo_producto (
π codigo_producto ( DETALLE_DE_VENTA ) ) ) )
```

SQL:

```
SELECT p.codigo, p.nombre
FROM   producto p
WHERE  NOT EXISTS (
         SELECT 1
         FROM   detalle_de_venta d
         WHERE  d.codigo_producto = p.codigo
       );
```

Resultado esperado: un único producto, que existe en el catálogo pero no tiene ventas registradas.

codigo	nombre
6	Pastel de vainilla con fresa (porción)

Consulta 4 — Intersección (operador ∩)

Descripción: encontrar los insumos que se usan tanto en la receta del pastel de chocolate con manjar como en la del brownie, para saber qué insumos son comunes a ambas preparaciones y deben mantenerse siempre en existencia. Se obtienen los insumos de cada receta y se intersecan.

Álgebra relacional:

```
T1 ← π codigo_insumo ( σ nombre = 'Pastel de chocolate con manjar (porción)' (
PRODUCTO ) ⋈ codigo = codigo_producto RECETA )
T2 ← π codigo_insumo ( σ nombre = 'Brownie' ( PRODUCTO ) ⋈ codigo = codigo_producto
RECETA )
π codigo, nombre ( INSUMO ⋈ codigo = codigo_insumo ( T1 ∩ T2 ) )
```

SQL:

```
SELECT i.codigo, i.nombre
FROM   insumo i
WHERE  i.codigo IN (
         SELECT r.codigo_insumo
         FROM   receta r
         JOIN   producto p ON p.codigo = r.codigo_producto
         WHERE  p.nombre = 'Pastel de chocolate con manjar (porción)')
  AND  i.codigo IN (
         SELECT r.codigo_insumo
         FROM   receta r
         JOIN   producto p ON p.codigo = r.codigo_producto
         WHERE  p.nombre = 'Brownie')
ORDER BY i.codigo;
```

Resultado esperado: cinco insumos presentes en ambas recetas.

codigo	nombre
3	Azúcar (kg)
4	Harina de trigo (kg)
5	Huevos (unidades)
6	Mantequilla (kg)
7	Chocolate en polvo (kg)

Consulta 5 — Agrupamiento y función de agregación (operador γ)

Descripción: calcular, por producto, las unidades vendidas y el ingreso generado (cantidad por precio unitario), mostrando solo los productos cuyo ingreso total es de al menos 10 dólares y ordenándolos de mayor a menor. Une detalle_de_venta con producto, agrupa por producto y filtra el resultado de la agregación (HAVING).

Álgebra relacional:

```
π nombre, unidades, ingreso ( σ ingreso ≥ 10 ( γ codigo, nombre; SUM(cantidad) →
unidades, SUM(cantidad · precio_unitario) → ingreso ( DETALLE_DE_VENTA ⋈
codigo_producto = codigo PRODUCTO ) ) )
```

SQL:

```
SELECT   p.nombre,
         SUM(d.cantidad)                     AS unidades,
         SUM(d.cantidad * d.precio_unitario) AS ingreso
FROM     detalle_de_venta d
JOIN     producto p ON p.codigo = d.codigo_producto
GROUP BY p.codigo, p.nombre
HAVING   SUM(d.cantidad * d.precio_unitario) >= 10
ORDER BY ingreso DESC;
```

Resultado esperado: los cinco productos que superan los 10 dólares de ingreso; el pastel de chocolate con manjar es el más rentable por el pedido de 12 porciones.

nombre	unidades	ingreso
Pastel de chocolate con manjar (porción)	13	45,50
Alfajor de chocolate	24	30,00
Alfajor de maicena	24	24,00
Brownie	7	14,00
Galleta de avena	12	12,00

7. CONCLUSIONES

Conclusión 1: modelado conceptual. La elaboración del diagrama E-R y su transformación al modelo relacional permitió comprender cómo un negocio real se traduce en entidades, atributos y relaciones. Las mayores dificultades estuvieron en decidir cómo representar la receta (una relación N:M con un atributo propio que se convirtió en la tabla receta), en tratar al pedido como entidad débil de la venta y en permitir ventas sin cliente registrado. Resolver estas decisiones antes de escribir código evitó rediseños posteriores y dejó un esquema de 11 tablas sin redundancias evidentes.

Conclusión 2: implementación SQL. Escribir el DDL y el DML ayudó a entender que el esquema no solo define tablas, sino también reglas de negocio: las llaves primarias y foráneas, UNIQUE, NOT NULL, DEFAULT y CHECK garantizan que los datos sean coherentes. Durante la carga de los datos de ejemplo, estas restricciones permitieron detectar inconsistencias, como llaves primarias duplicadas y referencias a productos o proveedores inexistentes, que se corrigieron antes de entregar el script. Además, se comprobó la importancia del orden de carga de las tablas y de elegir bien las acciones ON DELETE (RESTRICT frente a CASCADE).

Conclusión 3: álgebra relacional. Formular primero cada consulta en álgebra relacional obligó a definir con precisión qué relaciones intervienen, qué condición se aplica y qué atributos se proyectan, lo que hizo más directa su traducción a SQL. Las consultas más útiles para el negocio fueron la reunión de cliente,

venta y pedido (seguimiento de pedidos con su saldo pendiente) y la agrupación de ingresos por producto; la diferencia y la intersección, por su parte, mostraron cómo operaciones sobre conjuntos se expresan en SQL con NOT EXISTS e IN cuando el motor no dispone de INTERSECT.

8. BIBLIOGRAFÍA

[1] A. Silberschatz, H. F. Korth y S. Sudarshan, *Database System Concepts*, 7.ª ed. Nueva York, NY, EE. UU.: McGraw-Hill, 2019.

[2] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2016.

[3] E. F. Codd, «A relational model of data for large shared data banks,» *Communications of the ACM*, vol. 13, n.º 6, pp. 377–387, jun. 1970.

[4] P. P.-S. Chen, «The entity-relationship model—toward a unified view of data,» *ACM Transactions on Database Systems*, vol. 1, n.º 1, pp. 9–36, mar. 1976.

[5] Oracle Corporation, «MySQL 8.0 Reference Manual.» [En línea]. Disponible en: https://dev.mysql.com/doc/refman/8.0/en/. [Accedido: 8-nov-2026].

[6] Servicio de Rentas Internas (SRI), «Circular n.º NAC-DGECCGC25-00000006: Vigencia de la tarifa general del IVA,» Quito, Ecuador, 26-dic-2025.